

LAB 2. XÂY DỰNG LỚP DOMAIN & DEPENDENCY INJECTION

Ở buổi trước, chúng ta đã xây dựng xong "bộ khung xương" cho dự án. Hôm nay, chúng ta sẽ bắt đầu thêm "thịt và da" vào bộ khung đó. Chúng ta sẽ định nghĩa sản phẩm là gì, tạo ra một kho chứa sản phẩm (dưới dạng dữ liệu giả) và học một kỹ thuật cực kỳ quan trọng là **Dependency Injection** để kết nối các lớp một cách linh hoạt.

MỤC TIÊU BÀI HỌC

Kết thúc buổi học, bạn sẽ có thể:

1. **Tạo các lớp POCO** (Plain Old CLR Object) để định nghĩa các đối tượng nghiệp vụ cốt lõi.
2. **Thiết kế hợp đồng (contract)** bằng Interface để định nghĩa các hành vi, tách biệt logic nghiệp vụ khỏi cách triển khai cụ thể.
3. **Hiểu và áp dụng** được nguyên lý Dependency Injection (DI) - một trong những kỹ thuật nền tảng của phát triển phần mềm hiện đại.
4. **Sử dụng DI Container** có sẵn trong ASP.NET Core để đăng ký và "tiêm" (inject) các dịch vụ.
5. **Truyền dữ liệu** từ Controller sang View và dùng cú pháp Razor để hiển thị một danh sách động.

YÊU CẦU KIẾN THỨC

- Đã hoàn thành bài thực hành số 1.
- Kiến thức về Class và Interface trong C#.
- Hiểu biết cơ bản về vòng đời của một HTTP request trong mô hình MVC.

CÔNG CỤ CẦN THIẾT

- Solution SportsStore đã được tạo ở buổi 1.
- Visual Studio 2022 hoặc Visual Studio Code với C# Dev Kit.

NỘI DUNG

Phần 1: Phân tích lý thuyết

Trước khi code, hãy cùng làm rõ một vài khái niệm quan trọng.

1. POCO là gì?

POCO là viết tắt của **Plain Old CLR Object**. Đây là một thuật ngữ để chỉ những class C# "thuần túy".

- **"Thuần túy" nghĩa là gì?** Nó chỉ chứa dữ liệu (các thuộc tính) và không kế thừa từ một framework phức tạp nào. Nó không có logic về database, không có code về UI. Nó chỉ đơn giản là đại diện cho một khái niệm nghiệp vụ.
- **Ví dụ:** Product.cs của chúng ta sẽ là một POCO. Nó chỉ mô tả một sản phẩm trông như thế nào, chứ không quan tâm sản phẩm đó được lưu ở đâu.

2. Interface - Bản hợp đồng

Hãy tưởng tượng HomeController (ông chủ) muốn lấy danh sách sản phẩm. Thay vì đi thẳng đến một "nhà kho" cụ thể (ví dụ FakeProductRepository), ông chủ sẽ tạo ra một bản hợp đồng tên là IProductRepository.

- **Bản hợp đồng (IProductRepository)** này quy định: "Bất kỳ ai muốn làm nhà kho cho tôi, người đó phải có khả năng cung cấp một danh sách sản phẩm (IQueryable<Product> Products)."
- **Bên thực thi (FakeProductRepository):** Một nhà kho giả, ký vào bản hợp đồng và cam kết: "Tôi sẽ cung cấp danh sách sản phẩm như yêu cầu". Sau này, chúng ta có thể có một nhà kho khác (EfProductRepository) lấy từ database thật, cũng ký vào bản hợp đồng này.
- **Lợi ích:** Ông chủ (HomeController) chỉ cần làm việc với bản hợp đồng. Ông không quan tâm nhà kho là giả hay thật. Điều này giúp hệ thống trở nên **linh hoạt, dễ thay đổi và dễ kiểm thử (testing)**.

3. Dependency Injection (DI) - Người quản gia trung gian

Vậy ai sẽ là người quyết định đưa cho ông chủ HomeController nhà kho FakeProductRepository để sử dụng? Đó chính là **DI Container** - người quản gia thông thái.

- **Bước 1: Đăng ký (Registration):** Chúng ta nói với người quản gia (trong file Program.cs): "Này quản gia, nếu có ai hỏi xin một 'bản hợp đồng IProductRepository', đưa cho họ một 'nhà kho FakeProductRepository'."

- **Bước 2: Yêu cầu (Resolution/Injection):** Khi HomeController được tạo ra, nó sẽ nói: "Tôi cần một ai đó thực thi hợp đồng IProductRepository để làm việc."
- **Bước 3: Tiêm (Injection):** Người quản gia sẽ nghe thấy, tạo ra một đối tượng FakeProductRepository và "tiêm" nó vào cho HomeController thông qua hàm khởi tạo (constructor).

Bằng cách này, HomeController không cần phải tự mình tạo ra FakeProductRepository (new FakeProductRepository()). Nó hoàn toàn không biết về sự tồn tại của lớp này, giúp giảm sự phụ thuộc cứng (tight coupling) giữa các thành phần.

Phần 2: Hướng dẫn thực hành chi tiết

Mở lại solution SportsStore đã tạo ở buổi 1 và bắt đầu.

Bước 1: Xây dựng trong SportsStore.Domain

Đây là lớp chứa đựng trái tim nghiệp vụ của chúng ta.

1. Tạo Model Product.cs

- Trong **Solution Explorer**, chuột phải vào project SportsStore.Domain -> **Add** -> **Class....**
- Đặt tên file là Product.cs.
- Thêm các thuộc tính sau vào class:

```
// SportsStore.Domain/Product.cs

namespace SportsStore.Domain;

/// <summary>
/// Lớp POCO đại diện cho một sản phẩm trong cửa hàng.
/// </summary>
public class Product
{
    // Dấu ? cho biết thuộc tính này có thể là null (nullable).

    public int ProductID { get; set; }
    public required string Name { get; set; } // required đảm bảo thuộc
    tính này phải được khởi tạo
    public required string Description { get; set; }
```

```
public decimal Price { get; set; }  
public required string Category { get; set; }  
  
// Thêm thuộc tính imageUrl để lưu đường dẫn ảnh  
public string? imageUrl { get; set; }  
}
```

2. Tạo Interface IProductRepository.cs

- Chuột phải vào project SportsStore.Domain -> **Add -> New Item....**
- Chọn **Interface** và đặt tên là IProductRepository.cs.
- Định nghĩa "hợp đồng" như sau:

```
// SportsStore.Domain/IProductRepository.cs  
  
namespace SportsStore.Domain;  
  
/// <summary>  
/// Interface (hợp đồng) định nghĩa các hoạt động liên quan đến kho sản phẩm.  
/// Bất kỳ lớp nào muốn làm việc với dữ liệu sản phẩm đều phải tuân thủ hợp đồng này.  
/// </summary>  
public interface IProductRepository  
{  
    // IQueryable cho phép xây dựng các truy vấn phức tạp hơn (sắp xếp, lọc)  
    // trước khi thực sự lấy dữ liệu. Nó linh hoạt hơn IEnumerable.  
    IQueryable<Product> Products { get; }  
}
```

Bước 2: Triển khai trong SportsStore.Infrastructure

Lớp này sẽ cung cấp một triển khai cụ thể (dù là giả) cho hợp đồng mà chúng ta vừa định nghĩa.

1. Tạo lớp FakeProductRepository.cs

- Trong **Solution Explorer**, chuột phải vào project SportsStore.Infrastructure -> **Add -> Class....**
- Đặt tên file là FakeProductRepository.cs.

- Nội dung file:

```
// SportsStore.Infrastructure/FakeProductRepository.cs

// Cần using để có thể thấy được Product và IProductRepository
using SportsStore.Domain;

namespace SportsStore.Infrastructure;

/// <summary>
/// Một triển khai "giả" của IProductRepository.
/// Dùng để phát triển và kiểm thử mà không cần kết nối database thật.
/// </summary>
public class FakeProductRepository : IProductRepository // Ký vào "hợp đồng"
{
    // Triển khai thuộc tính Products mà hợp đồng yêu cầu.
    public IQueryable<Product> Products => new List<Product>
    {
        new Product {
            Name = "Football", Description = "FIFA-approved size and weight",
            Price = 25, Category = "Soccer", ImageUrl = "/images/football.png"
        },
        new Product {
            Name = "Surf Board", Description = "A board for riding the waves",
            Price = 179, Category = "Surfing", ImageUrl = "/images/surfboard.png"
        },
        new Product {
            Name = "Running Shoes", Description = "Comfortable and stylish running shoes",
            Price = 95, Category = "Running", ImageUrl = "/images/runningshoes.png"
        },
        new Product {
            Name = "Kayak", Description = "A boat for one person",
            Price = 275, Category = "Watersports", ImageUrl = "/images/kayak.png"
        },
    },
}
```

```
        new Product {
            Name = "Corner Flags", Description = "Give your playing field a
professional touch",
            Price = 34.95m, Category = "Soccer", ImageUrl =
"/images/cornerflags.png"
        }
    }.AsQueryable<Product>(); // Chuyển List thành IQueryable
}
```

Bước 3: Sử dụng trong SportsStore.WebUI

Đây là nơi chúng ta kết nối mọi thứ và hiển thị kết quả cho người dùng.

1. Đăng ký dịch vụ vào DI Container (Program.cs)

- Mở file Program.cs trong project SportsStore.WebUI.
- Tìm đến dòng `builder = WebApplication.CreateBuilder(args);`.
- Ngay bên dưới dòng `// Add services to the container.`, thêm dòng code sau:

```
// SportsStore.WebUI/Program.cs

// ... các dòng code có sẵn
builder.Services.AddControllersWithViews();

// -- BẮT ĐẦU THÊM CODE --
// Đăng ký dịch vụ với DI Container.
// Khi có một yêu cầu cho IProductRepository, hãy tạo một instance của
FakeProductRepository.
// AddScoped: Một instance mới sẽ được tạo cho mỗi HTTP request.
builder.Services.AddScoped<IProductRepository, FakeProductRepository>();
// -- KẾT THÚC THÊM CODE --

var app = builder.Build();
// ... các dòng code còn lại
```

- Bạn sẽ cần thêm các using ở đầu file để Visual Studio nhận diện được IProductRepository và FakeProductRepository:

```
using SportsStore.Domain;
using SportsStore.Infrastructure;
```

2. Inject IProductRepository vào HomeController

- Mở file HomeController.cs trong thư mục Controllers.
- Sửa lại class này như sau:

```
// SportsStore.WebUI/Controllers/HomeController.cs

using Microsoft.AspNetCore.Mvc;
using SportsStore.Domain; // Cần using để biết IProductRepository

namespace SportsStore.WebUI.Controllers;

public class HomeController : Controller
{
    // 1. Khai báo một biến private để lưu trữ repository
    private IProductRepository _repository;

    // 2. Tạo constructor nhận vào một IProductRepository
    // Đây là nơi DI Container sẽ "tiêm" FakeProductRepository vào.
    public HomeController(IProductRepository repo)
    {
        _repository = repo;
    }

    // 3. Sửa lại Action Index để lấy dữ liệu và truyền sang View
    public IActionResult Index()
    {
        // Lấy danh sách sản phẩm từ repository
        var products = _repository.Products;

        // Trả về View, kèm theo danh sách sản phẩm
        return View(products);
    }
}
```

3. Hiển thị danh sách sản phẩm bằng Razor (Index.cshtml)

- Mở file Index.cshtml trong thư mục Views/Home.
- Xóa toàn bộ nội dung mặc định và thay bằng code sau:

```
@* SportsStore.WebUI/Views/Home/Index.cshtml *@

@* 1. Khai báo model mà View này sẽ nhận: một danh sách các sản phẩm *@
```

```
@model IQueryable<Product>

@{
    ViewData["Title"] = "Products";
}

<h2 class="text-center my-4">Product List</h2>

<div class="row">
    @* 2. Dùng vòng lặp foreach để duyệt qua từng sản phẩm trong Model *@
    @foreach (var p in Model)
    {
        <div class="col-md-4 col-sm-6 mb-4">
            <div class="card h-100">
                @* Giả sử chúng ta sẽ có ảnh trong wwwroot/images *@
                
                <div class="card-body">
                    <h5 class="card-title">
                        @p.Name
                        <span class="badge bg-primary float-
end">@p.Price.ToString("c")</span>
                    </h5>
                    <p class="card-text">@p.Description</p>
                </div>
            </div>
        </div>
    }
</div>
```

Bước 4: Chạy thử ứng dụng

- Hãy chắc chắn rằng SportsStore.WebUI được chọn làm **Startup Project** (chữ in đậm trong Solution Explorer).
- Nhấn F5 hoặc nút "Run".
- Trình duyệt sẽ mở ra và bạn sẽ thấy một trang chủ hiển thị danh sách 5 sản phẩm mà chúng ta đã hard-code, chứng tỏ kiến trúc và Dependency Injection đã hoạt động hoàn hảo! (Ảnh có thể chưa hiện vì chúng ta chưa thêm file ảnh, nhưng các thông tin khác sẽ có đủ).

Phần 3: Tổng kết & Mở rộng

Kết quả

Bạn đã thành công xây dựng được luồng dữ liệu đầu tiên của ứng dụng:

1. **Domain** định nghĩa Product và hợp đồng IProductRepository.
2. **Infrastructure** cung cấp dữ liệu giả qua FakeProductRepository.
3. **WebUI** sử dụng **DI** để lấy dữ liệu từ HomeController và **Razor** để hiển thị nó lên View.
4. Bạn đã có một trang web động, hiển thị dữ liệu từ lớp logic thay vì nội dung HTML tĩnh.

Bài tập mở rộng

1. **Thêm sản phẩm:** Thêm 2-3 sản phẩm mới vào danh sách trong FakeProductRepository.cs và xem chúng tự động xuất hiện trên trang chủ.
2. **Tạo View Model:** Tạo một class ProductsListViewModel.cs trong SportsStore.WebUI/Models. Class này chứa một thuộc tính public IQueryable<Product> Products { get; set; }. Sửa lại HomeController và Index.cshtml để sử dụng ViewModel này thay vì truyền IQueryable<Product> trực tiếp. Đây là một thực hành tốt để giữ View độc lập hơn với Domain Model.
3. **Hiển thị Category:** Sửa đổi Index.cshtml để hiển thị cả thuộc tính Category của mỗi sản phẩm.

TÀI LIỆU THAM KHẢO

- [1]. Dependency Injection in ASP.NET Core: Microsoft Docs - Dependency injection in ASP.NET Core
- [2]. Razor syntax reference for ASP.NET Core: Microsoft Docs - Razor syntax
- [3]. IQueryable vs IEnumerable: Stack Overflow - What is the difference between IQueryable and IEnumerable?